Control: conditionals

⏪ Recap

- Variable: a name tag for a value; `=` assigns
- Function: arguments in, side effect and/or return value out
  - `def` to define your own; `return` to hand a value back
  - Scope: names stay inside their function
- Types: `str`, `int`, `float`; convert with `int()`, `float()`, `str()`
- String methods: `strip()`, `capitalize()`, `title()`

🤔 Scope and `return`: what gets printed?

```
def func1():
    x = 10
    y = 20
    return x + y

def func2(y):
    x = 40
    print(x + y)

a = func1()
print(a)

b = func2(2)
print(b)

x = 20
func2(x)
print(x)
```

Control

- **conditionals**: branching
- loops: repetition

1. Is x less than y?

```
Enter a number: 5
Enter another number: 3
x is greater than y

Enter a number: 2
Enter another number: 2
x is equal to y

Enter a number: 1
Enter another number: 4
x is less than y
```

Comparison operators

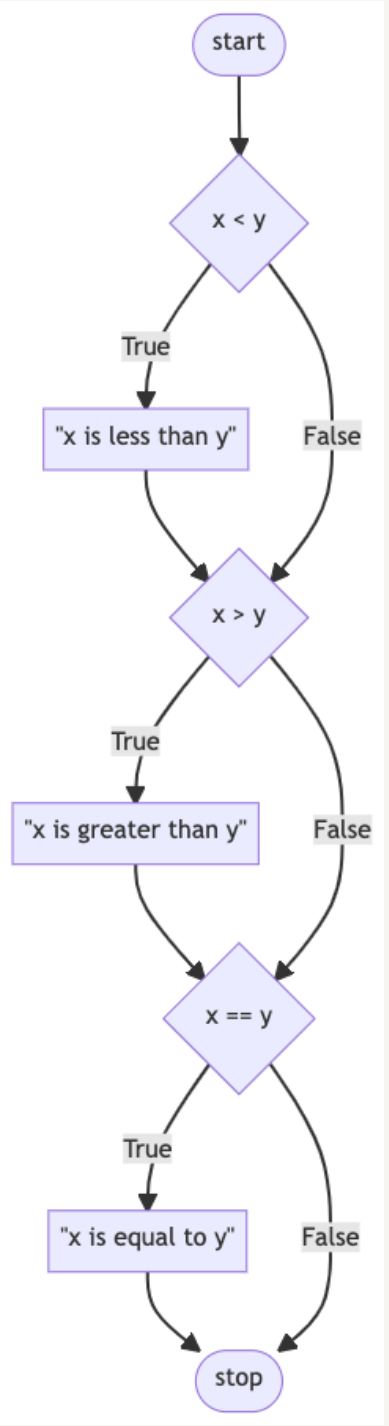
Operator	Meaning
`==`	equal to
`!=`	not equal to
`>`, `<`	greater than, less than
`>=`, `<=`	greater than or equal to, less than or equal to

- `=` **assigns.**
- `==` **compares.**

`if` to run code only when a condition is true

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

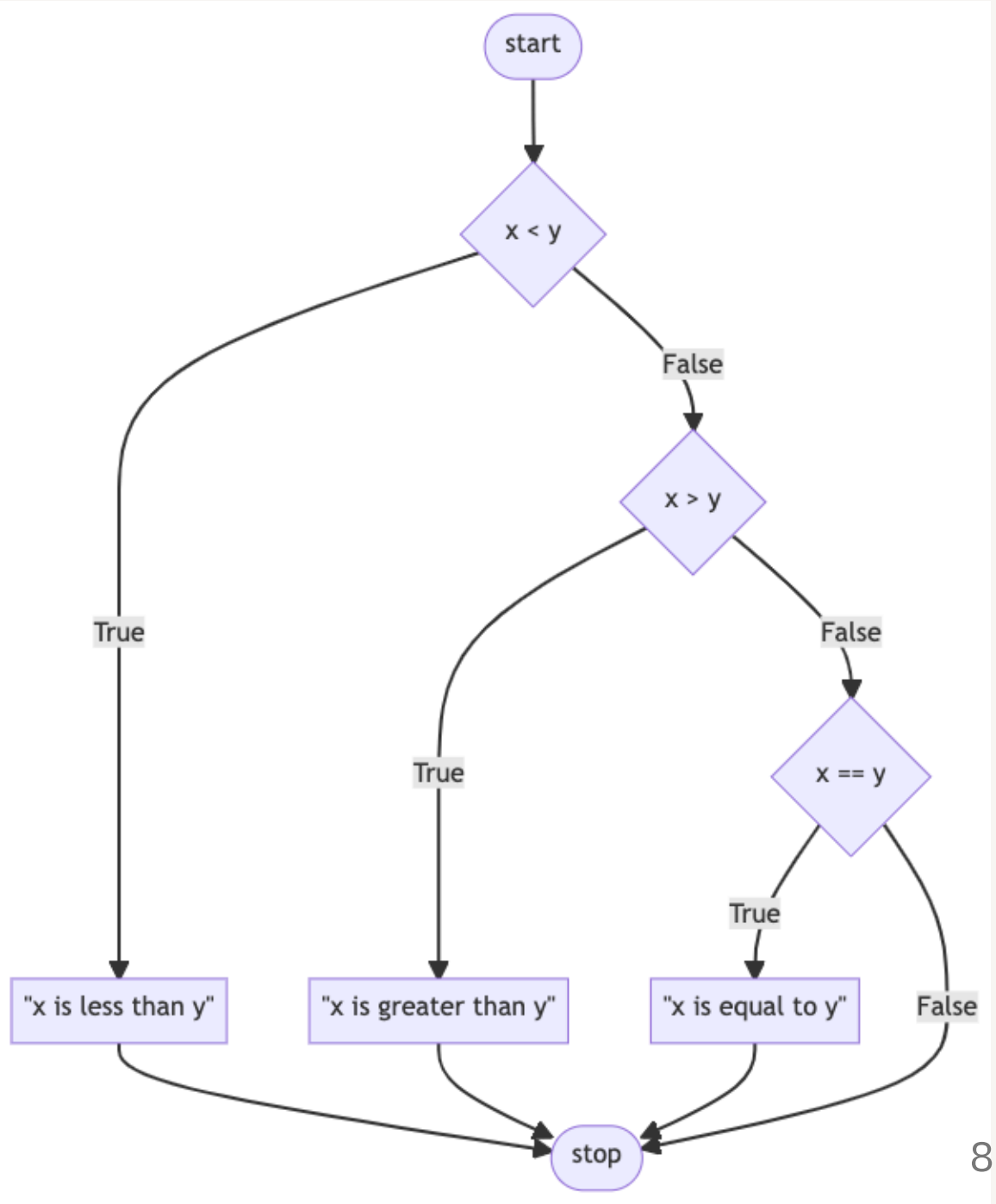
if x < y: # if, a condition, and a colon
    print("x is less than y") # body
if x > y:
    print("x is greater than y")
if x == y:
    print("x is equal to y")
```



`elif` (else if) to check only when earlier conditions were false

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

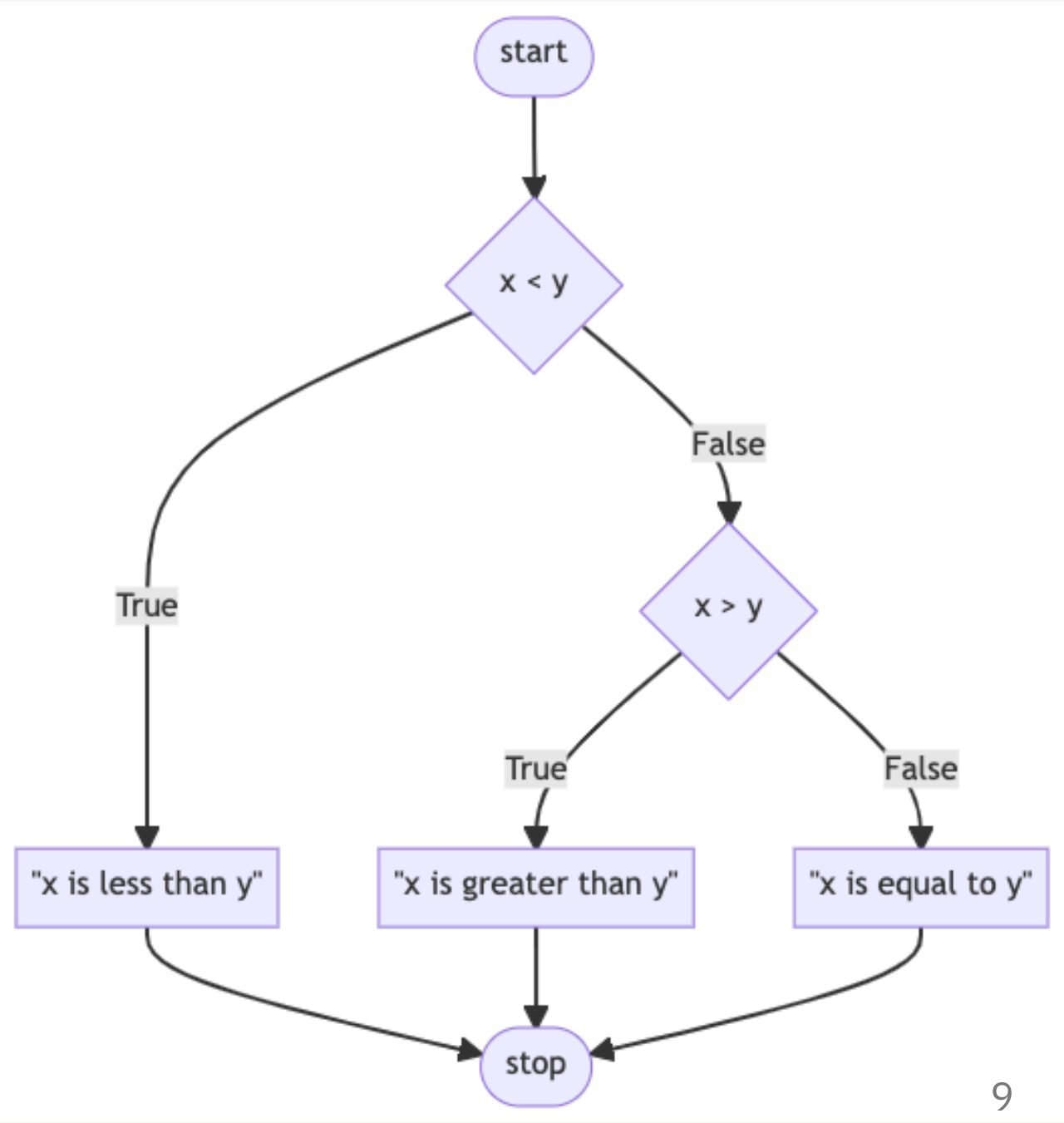
if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
elif x == y:
    print("x is equal to y")
```



`else` to catch everything left over

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x < y:
    print("x is less than y")
elif x > y:
    print("x is greater than y")
else:
    print("x is equal to y")
```



Nested `if` statements

```
x = int(input("Enter a positive number: "))
y = int(input("Enter another number: "))

if x > 0:
    if x < y:
        print("x is less than y")
    elif x > y:
        print("x is greater than y")
    else:
        print("x is equal to y")
else:
    print("x is not positive")
```

💻 2. Find the smallest number

Requirements:

- Prompt the user to enter three different numbers, convert them to integers, and store them in variables named x, y, and z.
- Use **nested if** statements to find the smallest number among the three.
- Print out which variable holds the smallest value with a message like "x is the smallest."

Expected Outputs:

```
Enter a number (x): 5
Enter another number (y): 3
Enter another number (z): 7
y is the smallest
```

3. Is x equal to y?

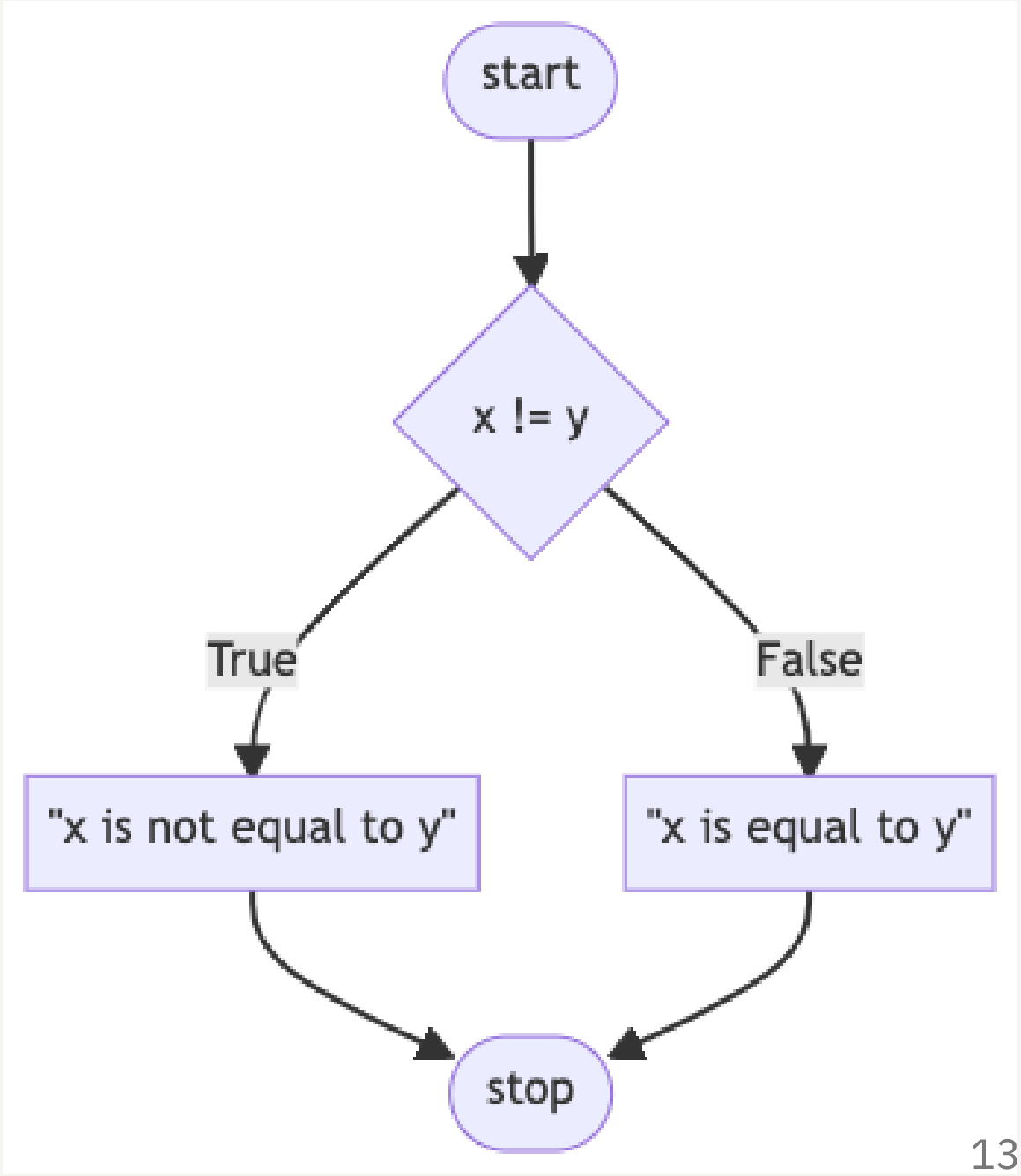
```
Enter a number: 5
Enter another number: 3
x is not equal to y

Enter a number: 7
Enter another number: 7
x is equal to y
```

!= (not equal to)

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x != y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```



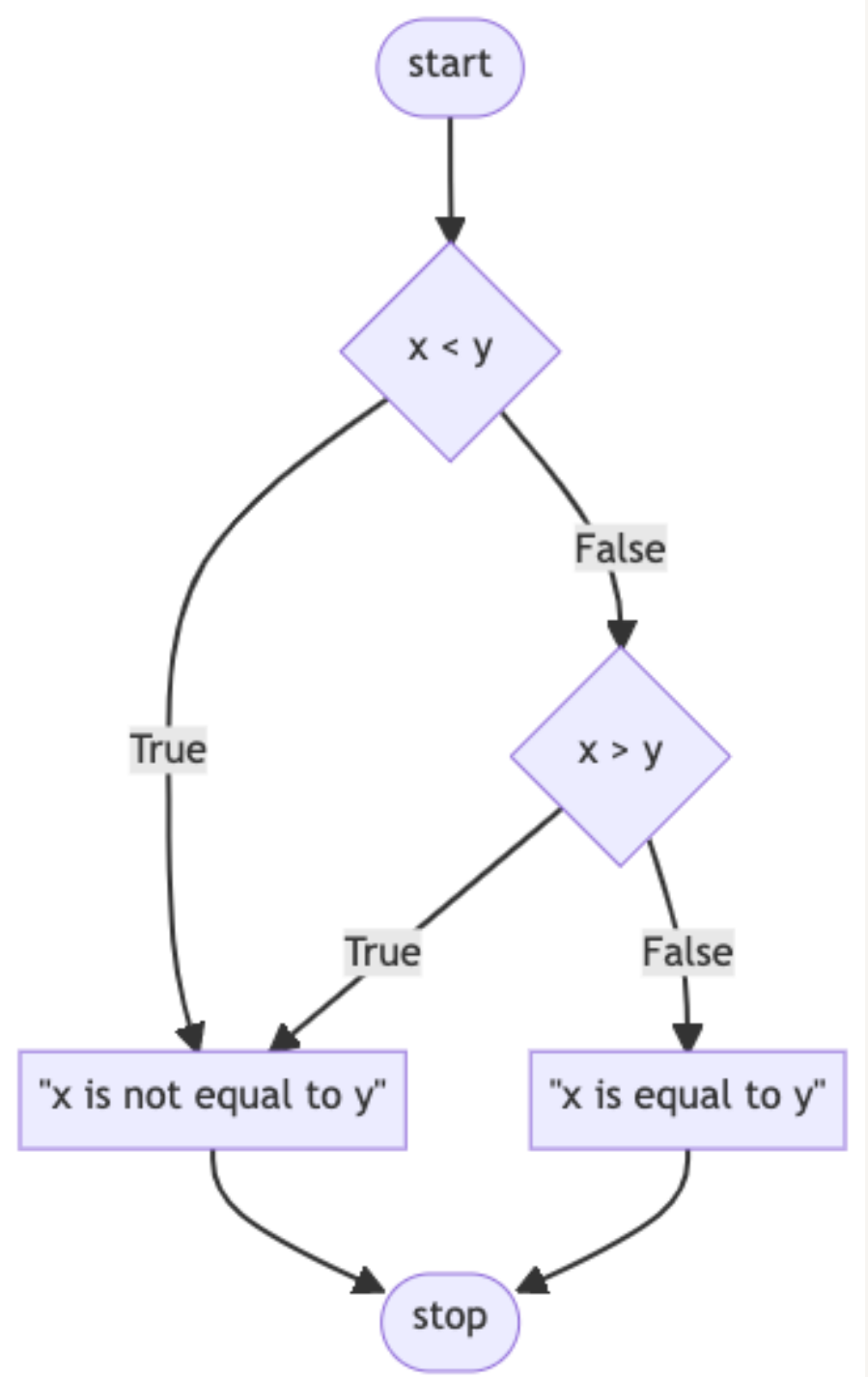
Logical operators to combine conditions

- `or` : either A or B is true
- `and` : both A and B are true
- `not` : negates the truth value

`or` to check if either condition is true

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if x < y or x > y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```



`not` to negate a condition

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))

if not x == y:
    print("x is not equal to y")
else:
    print("x is equal to y")
```

4. Gradebook

```
Enter your score: 87
A

Enter your score: 72
B

Enter your score: 60
C
```

`and` to check if both conditions are true

`and`: two conditions

```
score = int(input("Enter your score: "))

if score >= 85 and score <= 100:
    print("A")
elif score >= 80 and score < 85:
    print("A-")
elif score >= 75 and score < 80:
    print("B+")
elif score >= 70 and score < 75:
    print("B")
elif score >= 65 and score < 70:
    print("B-")
else:
    print("C")
```

Compound inequality: the same check

```
score = int(input("Enter your score: "))

if 85 <= score <= 100:
    print("A")
elif 80 <= score < 85:
    print("A-")
elif 75 <= score < 80:
    print("B+")
elif 70 <= score < 75:
    print("B")
elif 65 <= score < 70:
    print("B-")
else:
    print("C")
```

One bound is enough: `elif` already ruled out the higher grades

```
score = int(input("Enter your score: "))

if score >= 85:
    print("A")
elif score >= 80:   # we already know score < 85 from above
    print("A-")
elif score >= 75:   # score < 80
    print("B+")
elif score >= 70:   # score < 75
    print("B")
elif score >= 65:   # score < 70
    print("B-")
else:               # score < 65
    print("C")
```

🤔 A score of 90: what gets printed?

```
score = int(input("Enter your score: "))

if score >= 85:
    print("A")
if score >= 80:
    print("A-")
if score >= 75:
    print("B+")
if score >= 70:
    print("B")
if score >= 65:
    print("B-")
```

💻 5. Movie ticket

Requirements:

Use `input()` to ask for the customer's age (convert it to an integer) and the day of the week.

Print the ticket price:

- Tuesday: `$6` for everyone
- Otherwise, children (under 13) and seniors (65 or older): `$8`
- Everyone else: `$12`

Expected Outputs:

```
Enter your age: 30
Enter the day: Tuesday
$6

Enter your age: 70
Enter the day: Friday
$8

Enter your age: 13
Enter the day: Saturday
$12
```

6. Parity check

```
Enter a number: 4
x is even

Enter a number: 7
x is odd
```

`%` (modulo) to get the remainder

```
x = 4
print(x % 2) # 0

x = 7
print(x % 2) # 1

x = 10
print(x % 8) # 2
```

What must `is_even()` return?

```
def main():
    x = int(input("Enter a number: "))

    x_is_even = is_even(x)

    if x_is_even:
        print("x is even")
    else:
        print("x is odd")

def is_even(x):
    ...

main()
```

Conditions evaluate to `True` or `False`

```
print(2 > 1)            # True
print(type(True))       # <class 'bool'>

if True:
    print("always get printed")

if False:
    print("never get printed")
```

Each condition becomes `True` or `False` before `if` checks it

```
if 2 > 1:
    print("A")
else:
    print("B")
```

```
if True:                # 2 > 1
    print("A")
else:
    print("B")
```

```
if 2 > 1 or 2 == 1:
    print("A")
else:
    print("B")
```

```
if True or False:   # 2 > 1 or 2 == 1
    print("A")
else:
    print("B")
```

🤔 What gets printed?

```
x = 7

print(x > 5)                    # 1
print(x % 2 == 0)               # 2
print(x > 5 and x % 2 == 0)     # 3

result = x < 10
print(result)                   # 4

if result:
    print("small")              # 5
```

Return the condition directly: no `if` needed

```
def is_even(x):
    if x % 2 == 0:
        return True
    else:
        return False
```

```
def is_even(x):
    return x % 2 == 0  # True or False
```

```
print(is_even(10))    # True
print(is_even(7))     # False
```

No variable needed: call `is_even()` inside the `if`

```
def main():
    x = int(input("Enter a number: "))

    if is_even(x):           # no need for x_is_even variable
        print("x is even")
    else:
        print("x is odd")

def is_even(x):
    return x % 2 == 0        # True or False

main()
```

💻 7. Days in a month

Requirements:

- Write `is_leap(year)`, which returns `True` if `year` is a leap year and `False` otherwise:
  - A year divisible by 4 is a leap year,
  - except a year divisible by 100, which isn't,
  - unless it's also divisible by 400, which is.
- Write `days_in_month(year, month)`, which returns the number of days in that month:
  - February (`2`): 29 in a leap year, 28 otherwise. Call `is_leap()`.
  - April, June, September, November (`4`, `6`, `9`, `11`): 30
  - Any other month: 31

Expected Outputs:

```
Enter a year: 2024
Enter a month: 2
Month 2 of 2024 has 29 days
```

```
Enter a year: 1900
Enter a month: 2
Month 2 of 1900 has 28 days
```

Wrap up

- `if` / `elif` / `else`: run at most one branch; later conditions are checked only if earlier ones were false
- Comparison operators: `==`, `!=`, `>`, `<`, `>=`, `<=` (`=` assigns, `==` compares)
- Logical operators: `and`, `or`, `not`
- `bool`: conditions evaluate to `True` or `False`
- `%` (modulo): the remainder; `x % 2 == 0` checks for even
- `return` exits a function; a function with no `return` hands back `None`